

3.2 — Create and Configure Custom Slash Commands and Skills

Concept

Custom slash commands extend Claude Code's built-in `/` commands with project-specific or user-specific automation. They are Markdown files that contain a prompt template, an optional YAML frontmatter block for configuration, and the `$ARGUMENTS` placeholder where runtime input is injected.

Scope: Project vs User

Project scope — stored in `.claude/commands/` at the repository root. These commands are available to every contributor who opens the project in Claude Code. Use project-scoped commands for workflows that are tightly coupled to the codebase: running the test suite, applying a code review checklist, or generating release notes from a commit range.

User scope — stored in `~/.claude/commands/` in the user's home directory. These commands travel with the developer across all projects. Use user-scoped commands for personal productivity patterns: a preferred commit message template, a debugging scaffold, or a refactoring sequence you apply everywhere.

When both scopes define a command with the same filename, the project-scoped command takes precedence, allowing teams to override personal defaults when working inside a specific repository.

YAML Frontmatter Options

A command file may open with a fenced YAML block that controls execution behavior. Three options appear most frequently on the exam.

context: fork — Spawns an isolated sub-agent context for the command. The forked context receives a copy of the current conversation state but writes no changes back to the parent. Use this when the command might consume many tokens, explore speculative branches, or produce output that should not pollute the main session (e.g., a deep-dive security audit or a large-scale refactor preview).

allowed-tools — Restricts which tools the command is permitted to invoke, expressed as a list. This is a least-privilege enforcement mechanism. A documentation command that only needs to read files should declare **allowed-tools: [Read]** so it cannot accidentally write, execute code, or call external APIs. Omitting **allowed-tools** grants the command access to all tools available in the current session.

argument-hint — A short string displayed in the command palette as a placeholder label, guiding the user on what to pass as `$ARGUMENTS`. It has no effect on execution; it is purely informational UX.

Skills as Reusable Knowledge Packages

A **skill** is a slash command whose primary payload is a structured knowledge document rather than an imperative prompt. Where a regular command says "do X with \$ARGUMENTS", a skill says "here is everything

you need to know about domain Y — now apply it." Skills are typically stored in a dedicated subdirectory (e.g., `.claude/commands/skills/`) and invoked by name. The Claude Certified Architect exam treats skills as a higher-order abstraction: they encode team conventions, domain ontologies, or architectural patterns that agents can load on demand.

Code Example

Project-scoped command: `.claude/commands/review.md`

```
---
context: fork
allowed-tools:
  - Read
  - Grep
argument-hint: "branch or PR number to review"
---
```

You are a senior engineer performing a focused code review.

Review all changes introduced in \$ARGUMENTS against the main branch.

Checklist:

1. Immutability – no in-place mutation of shared state
2. Error handling – every error path is explicit and logged
3. Input validation – all external data is validated at the boundary
4. Security – no hardcoded secrets, no unparameterized queries
5. Test coverage – new logic has corresponding unit tests

Return findings grouped by severity: CRITICAL, HIGH, MEDIUM, LOW.

Invoked as: `/project:review feature/auth-refactor`

User-scoped command: `~/claude/commands/commit.md`

```
---
allowed-tools:
  - Bash
argument-hint: "optional scope (e.g. auth, api, ui)"
---
```

Generate a conventional commit message for the currently staged changes.

Scope override (if provided): \$ARGUMENTS

Rules:

- Type must be one of: feat, fix, refactor, docs, test, chore, perf, ci
- Subject line ≤ 72 characters, imperative mood, no trailing period
- Add a body paragraph if the change is non-trivial

Run ``git diff --cached`` to inspect staged content, then output only the commit message – nothing else.

Invoked as: `/commit auth`

Skill: `.claude/commands/skills/api-conventions.md`

```
---
allowed-tools: []
argument-hint: "leave blank – load conventions into context"
---

# API Response Conventions (Skill)

All API responses in this project use a standard envelope:

- `success` (boolean) – true on 2xx, false otherwise
- `data` (any | null) – payload on success, null on error
- `error` (string | null) – human-readable message on failure, null on success
- `meta` (object | null) – pagination metadata when applicable: `total`, `page`, `limit`

When generating or reviewing API handlers, enforce this structure on every
response path including error branches. Do not return raw data without the envelope.
```

Common Mistakes

- **Placing commands in the wrong directory** — A file in `.claude/commands/` inside a subdirectory other than the project root will not be discovered. The path must be exactly `<project-root>/.claude/commands/<name>.md`.
- **Omitting `context: fork` for expensive commands** — Long-running or exploratory commands that do not fork will consume the parent session's context window and may introduce noise into the ongoing conversation.
- **Over-permissive `allowed-tools`** — Leaving `allowed-tools` unset on a read-only documentation command silently grants write and execution access. Always declare the minimal tool set.
- **Confusing `argument-hint` with validation** — `argument-hint` is a UI label only. It does not enforce argument format or presence. Validate within the prompt body if the argument is required.
- **Treating skills as commands** — Skills are knowledge payloads, not action prompts. A skill that ends with "now do X" will behave like a command and may produce unexpected side effects when invoked without a

clear target. Keep skills descriptive and let the calling context supply the action.

References

- Claude Code documentation: [Custom slash commands](#)
- Claude Code documentation: [Skills](#)
- Exam domain 3: Customization and Extensibility
- Related handouts: 3.1 (CLAUDE.md memory files), 3.3 (hooks system)